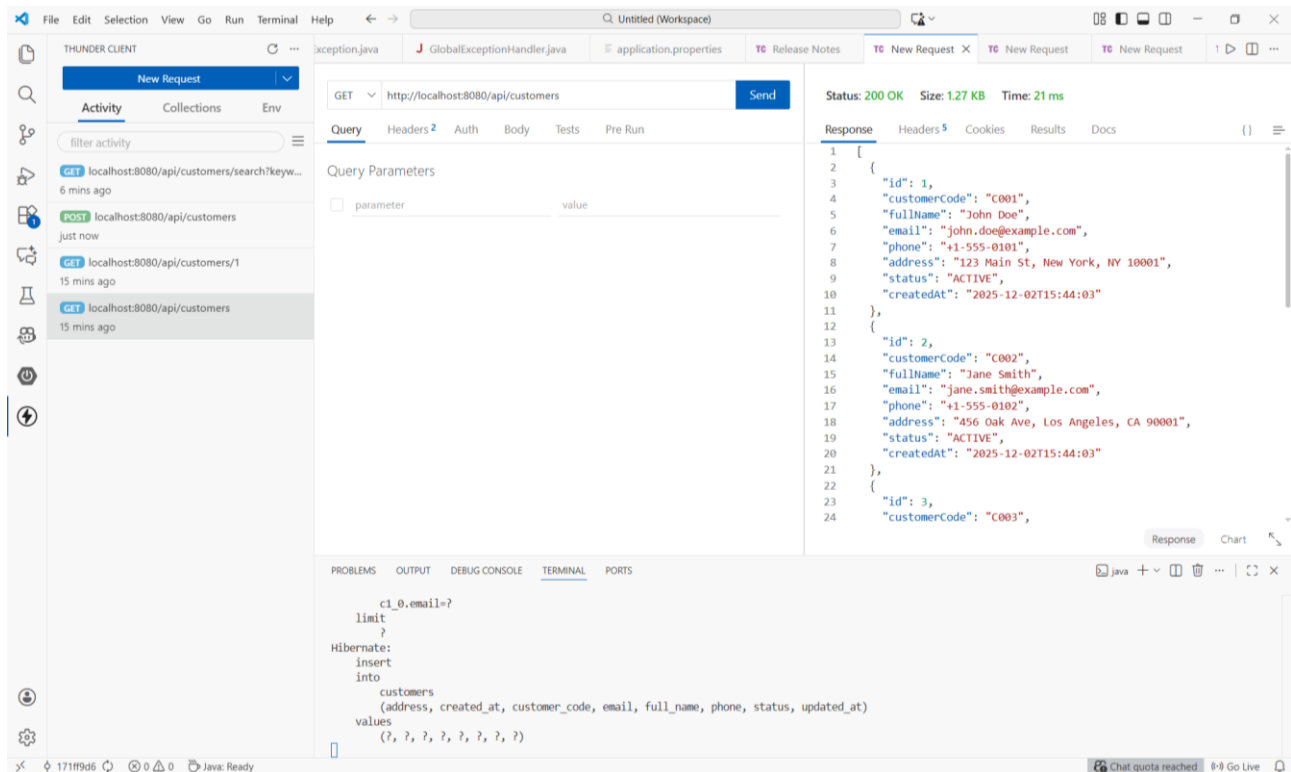


WEB APPLICATION DEVELOPMENT

LAB 8 - INCLASS

Test 1: GET All Customers

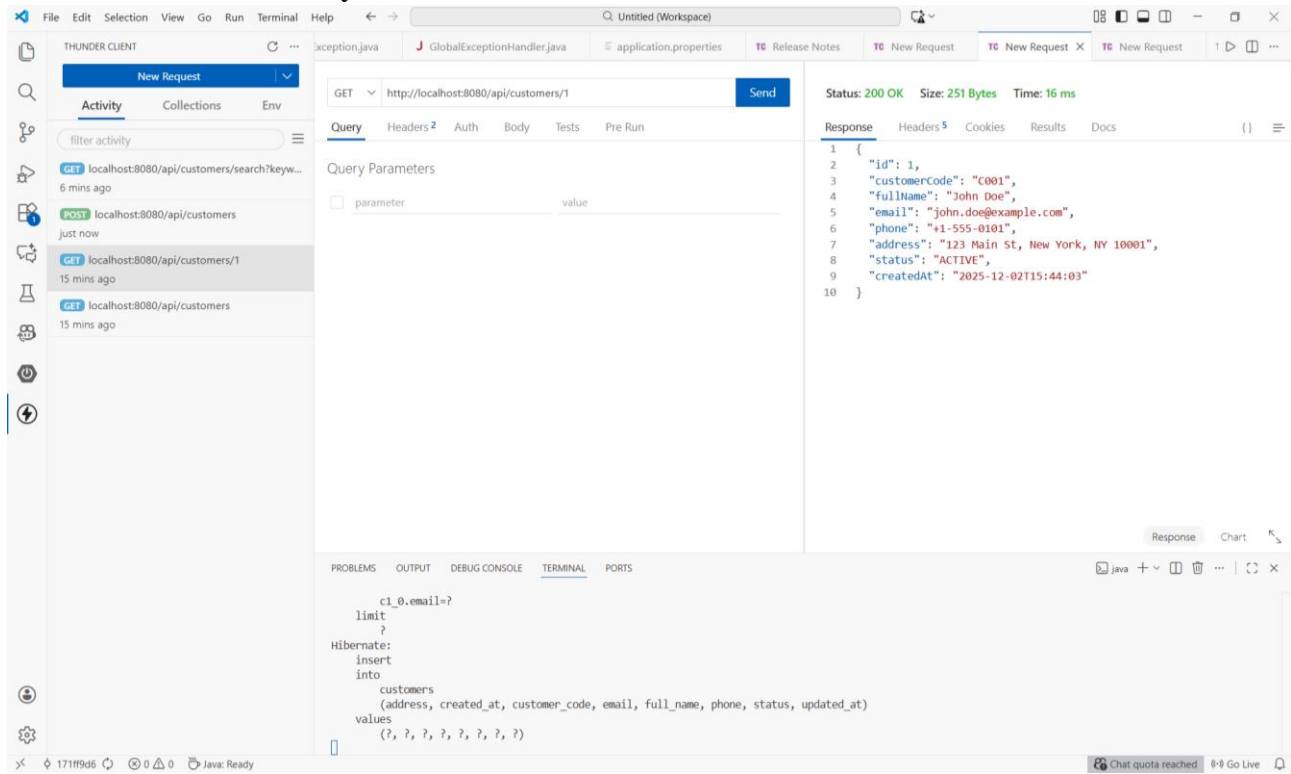


Explanation of the code flow of get all customers:

1. Browser / Frontend (Thunder Client) → Sends HTTP GET request such as GET `/api/customers` to `CustomerRestController.java`.
2. `CustomerRestController` receives request (`@GetMapping`)
 - Spring Boot detects the route `/api/customers`
 - Finds the matching controller method
 - Automatically maps the request
3. Controller calls service layer (`customerService.getAllCustomers()`)
 - Controller does not directly access the DB.
 - It delegates to the Service Layer.
4. Service calls `customerRepository.findAll()`
 - This is the JPA Repository method.
 - Spring Data JPA turns this into a SQL query: `SELECT * FROM customers;`
5. JPA executes SQL query automatically
 - Hibernate/JPA communicates with your database.
 - Fetches all rows from the customers table.
 - Converts each row → Customer entity object.
6. Repository returns `List<Customer>` to Service: Return Java objects: `List<Customer>`
7. Service converts Entity → DTO
 - Prevents exposing entity directly
 - Adds status, createdAt, etc.
 - Then service returns: `List<CustomerResponseDTO>`
8. Controller wraps DTO list into `ResponseEntity`: `return ResponseEntity.ok(customers);`
 - Sets status 200 OK
 - Prepares JSON response

9. Spring Boot automatically converts DTO list → JSON
10. JSON is returned to frontend
 - Thunder Client receives the JSON data
 - Frontend displays customer list

Test 2: GET Customer by ID

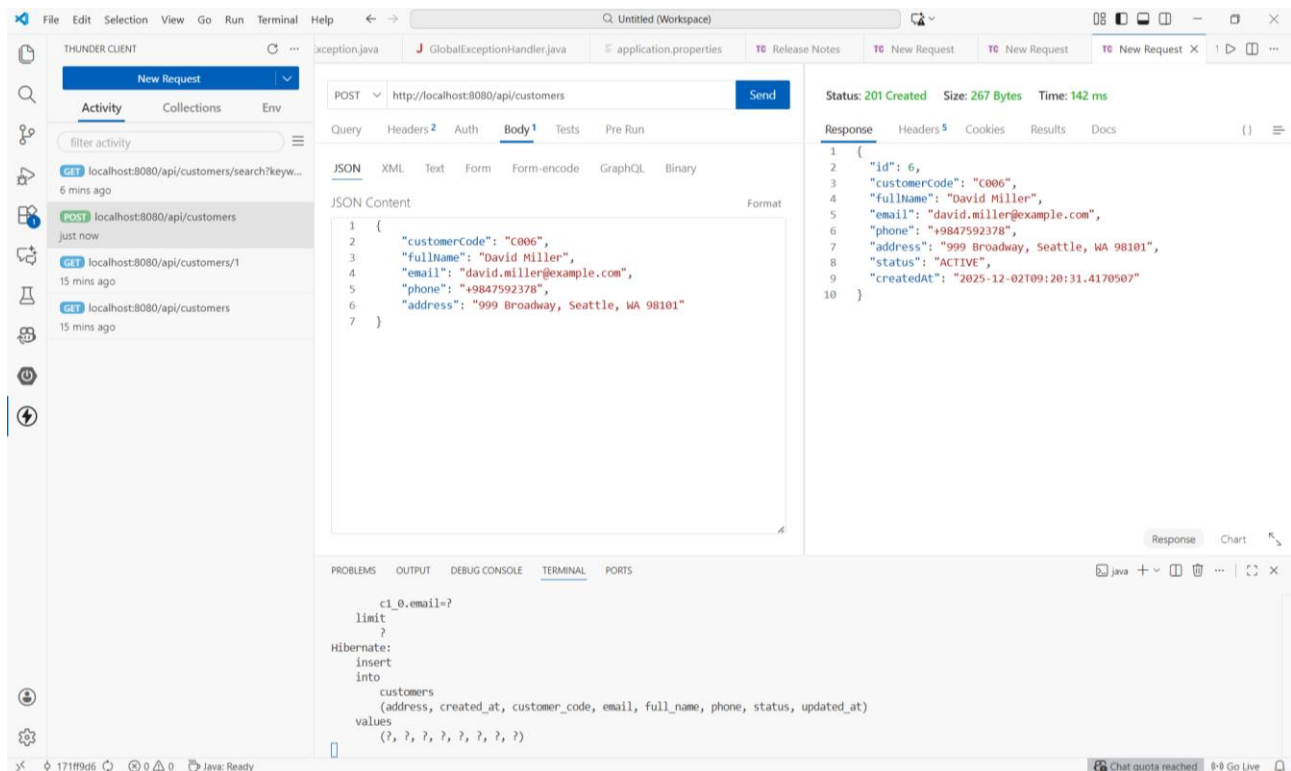


Explanation of the code flow of get customer by ID:

1. Browser / Frontend → sends HTTP GET request: Frontend (Thunder Client) calls: GET `/api/customers/5`. 5 is the customer ID the user wants to fetch.
2. CustomerRestController receives the request (`@GetMapping("/{id}")`). Spring Boot detects:
 - URL: `/api/customers/5`
 - Matches `/id` pattern
 - Extracts `id = 5`
 - Calls the method with that value
3. Controller calls `customerService.getCustomerById(id)`
 - Controller does not access DB directly.
 - Delegates work to a Service Layer.
4. Service calls `customerRepository.findById(id)`
 - JPA converts to SQL Query: `SELECT * FROM customers WHERE id = 5;`
5. Repository returns result
 - Customer exists → returns `Optional<Customer>`: Service unwraps `Optional` → gets Customer entity object.
 - Customer does not exist → `Optional` is empty. Then flow jumps to `GlobalExceptionHandler` → returns: 404 Not Found.
6. If customer found → Service converts Entity → DTO (`id`, `customerCode`, `fullName`, `email`, `phone`, `address`, `status`, `createdAt`)
7. Controller receives DTO and returns `ResponseEntity`. Spring sets
 - HTTP 200 OK
 - Body = `CustomerResponseDTO`

8. Spring Boot converts DTO → JSON automatically
9. JSON returned to frontend (Thunder Client will display it)

Test 3: POST Create Customer

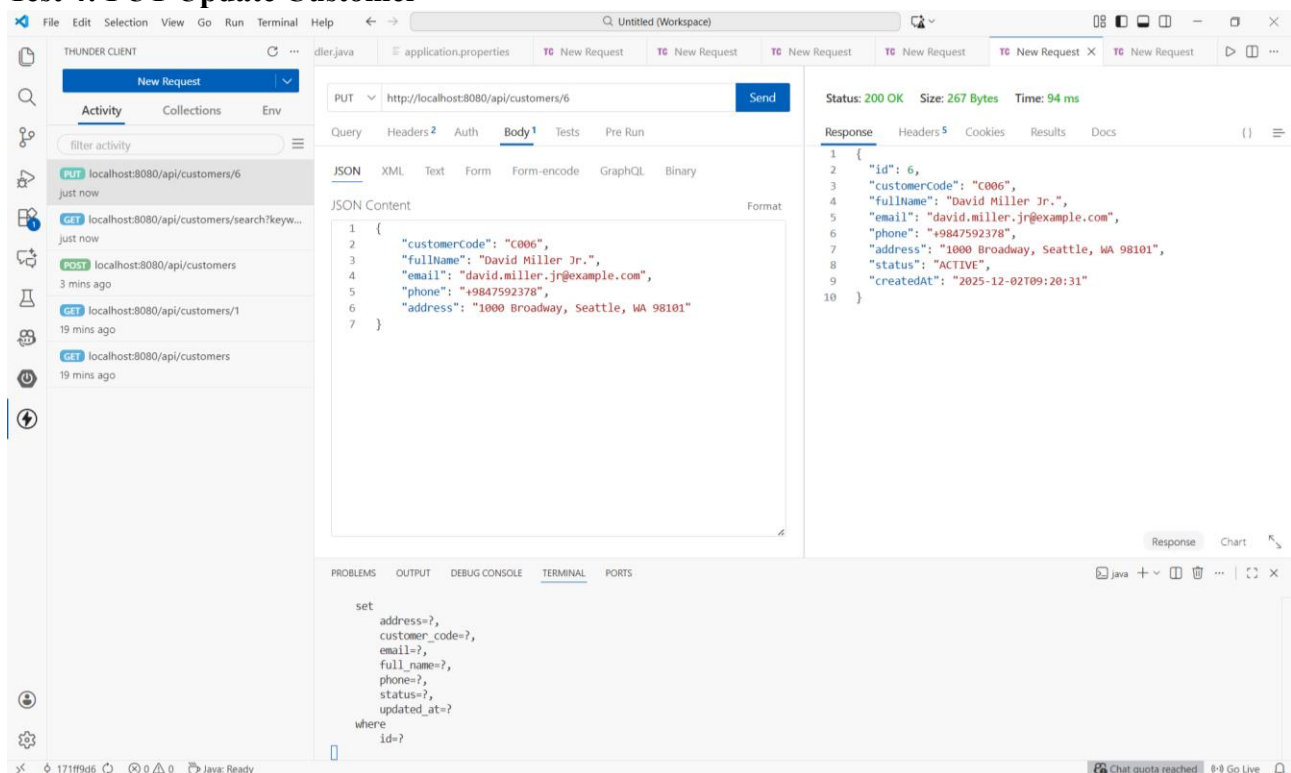


Explanation of the code flow of insert new customer:

1. Browser / Frontend → sends HTTP POST request
 - The frontend (Thunder Client) sends a POST request to: `POST /api/customers` with JSON body.
2. CustomerRestController receives the request (`@PostMapping`). Spring Boot automatically:
 - Reads JSON body
 - Converts it → `CustomerRequestDTO`
 - Runs validation (`@Valid`)
 - Calls the controller method
 - If validation fails → goes to `GlobalExceptionHandler` → returns 400 Bad Request.
3. Controller calls `customerService.createCustomer(requestDTO)`: Controller does NOT touch database.
4. Service checks for duplicate customerCode. If duplicate → `GlobalExceptionHandler` → returns: 409 Conflict
5. Service checks for duplicate email. If duplicate, returns 409 Conflict.
6. Service converts DTO → Entity
 - This step prepares the Customer object for saving.
 - Before saving:
 - + `createdAt` and `updatedAt` are null (will be set during `@PrePersist`)
 - + `status = ACTIVE` (default from entity)
7. Service calls `customerRepository.save(customer)`. JPA
 - Runs SQL insert
 - `@PrePersist` executes: `this.createdAt = LocalDateTime.now();`, `this.updatedAt = LocalDateTime.now();`
 - After saving:

- + Database generates id (auto increment)
- + Entity is updated with the generated ID
- + Entity is returned back to the Service
- 8. Service converts Entity → Response DTO (id, customerCode, fullName, email, phone, address, status, createdAt)
- 9. The controller returns ResponseEntity with HTTP 201 CREATED
 - Status: 201 Created
 - Body: JSON DTO
 - No HTML rendering → pure JSON
- 10. Spring Boot (Jackson) converts DTO → JSON
- 11. JSON response returned to Frontend (Thunder Client).
 - Frontend (Thunder Client) receives JSON and updates UI (e.g., refresh customer list, show success message).

Test 4: PUT Update Customer

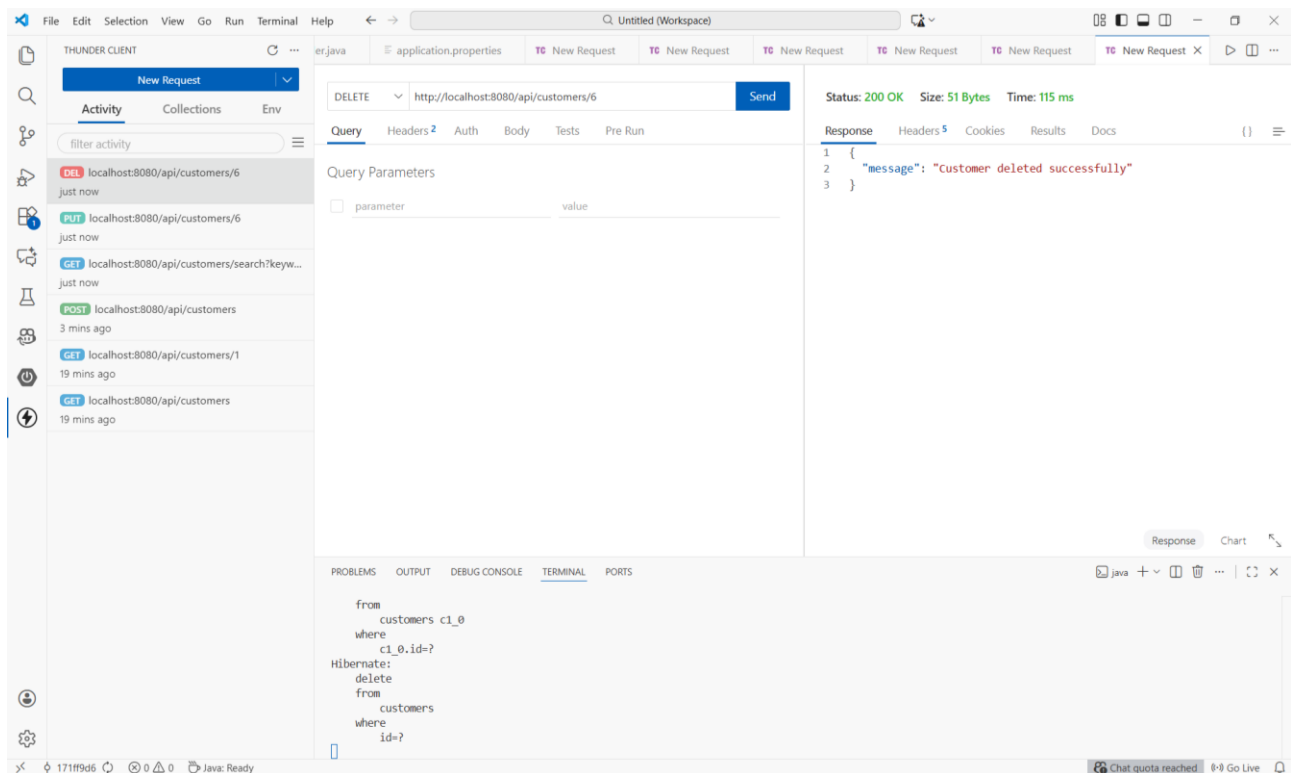


Explanation of the code flow of updated customer:

1. Browser / Frontend (Thunder Client) → sends HTTP PUT request. Frontend calls: PUT /api/customers/5 with JSON body.
2. CustomerRestController receives the request (@PutMapping("/{id}")). Spring Boot does:
 - Extract ID from URL → id = 5
 - Convert JSON → CustomerRequestDTO
 - Validate it with @Valid
 - Call controller method
 - If validation fails → 400 Bad Request (handled by GlobalExceptionHandler).
3. Controller calls customerService.updateCustomer(id, requestDTO). Controller does not work with entities or DB. It simply forwards the request.
4. Service loads existing customer from database
 - If ID does not exist → throw exception → GlobalExceptionHandler → return: 404 Not Found
 - If exists → continue.

5. Service checks for duplicate email (if changed)
 - If user tries to change the email, but the new email already belongs to another customer
 - Throw `DuplicateResourceException`
 - `GlobalExceptionHandler` returns: 409 Conflict
6. Service updates allowed fields
7. Service saves updated entity. JPA generates SQL for updating. Before updating fields: `@PreUpdate` runs `this.updatedAt = LocalDateTime.now();`.
8. Service converts Entity → Response DTO. DTO contains new updated values and unchanged fields like `customerCode`, `createdAt`, `status`.
9. Controller returns `ResponseEntity` with HTTP 200 OK
 - Status: 200 OK
 - Body: JSON DTO of updated customer
10. Spring Boot converts DTO → JSON
11. Frontend (Thunder Client) receives JSON and updates the UI

Test 5: DELETE Customer



Explanation of the deletion of customer:

1. Browser / Frontend → sends HTTP DELETE request. Frontend executes: `DELETE /api/customers/8`.
2. `CustomerRestController` receives request (`@DeleteMapping("/{id}")`). Spring Boot:
 - Extracts `id = 8` from the URL
 - Calls the controller method
3. Controller calls `service.deleteCustomer(id)`: `customerService.deleteCustomer(id)`;
 - The controller does not do any logic.
 - It simply passes the ID to the service layer.
4. Service checks if customer exists
 - If not found → throw `ResourceNotFoundException` → `GlobalExceptionHandler` → return: 404 Not Found
 - If found → continue to deletion

5. Service deletes customer
 - JPA generates SQL: DELETE FROM customers WHERE id = 8;
 - This removes the row permanently from the database.
 - No DTO conversion is needed because DELETE returns a simple message, not a Customer object.
6. Controller prepares success response
7. Spring Boot returns JSON to front-end with HTTP 200 OK

Test 6: Search Customers

The screenshot shows the Thunder Client interface. The 'Activity' pane on the left lists several requests. The selected request is a GET request to `localhost:8080/api/customers/search?keyword=john`. The 'Query' pane shows the query parameters: `keyword=john`. The 'Response' pane shows the JSON response:

```
[
  {
    "id": 1,
    "customerCode": "C001",
    "fullName": "John Doe",
    "email": "john.doe@example.com",
    "phone": "+1-555-0101",
    "address": "123 Main St, New York, NY 10001",
    "status": "ACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  },
  {
    "id": 3,
    "customerCode": "C003",
    "fullName": "Bob Johnson",
    "email": "bob.johnson@example.com",
    "phone": "+1-555-0103",
    "address": "789 Pine Rd, Chicago, IL 60601",
    "status": "ACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  }
]
```

The terminal at the bottom shows the SQL query generated by JPA:

```
c1_0.full_name,
c1_0.phone,
c1_0.status,
c1_0.updated_at
from
customers c1_0
where
lower(c1_0.full_name) like lower(concat('%', ?, '%')) escape ''
or lower(c1_0.email) like lower(concat('%', ?, '%')) escape ''
or lower(c1_0.customer_code) like lower(concat('%', ?, '%')) escape ''
```

Explanation of the code flow of searching customers with keyword:

1. Browser / Frontend → sends HTTP GET request with keyword such as GET `/api/customers/search?keyword=john`.
2. CustomerRestController receives request (`@GetMapping("/search")`). Spring Boot:
 - Detects the route `/api/customers/search`
 - Reads query parameter → `keyword = john`
 - Calls the controller method
3. Controller calls `service.searchCustomers(keyword)`.
 - Controller again does not perform any logic.
 - Delegates the processing to the Service Layer.
4. Service calls `repository.searchCustomers(keyword)`
5. JPA generates SQL query behind the scenes
 - This allows case-insensitive searching.
 - Matches anywhere in the text using `%keyword%`.
6. Database returns matching Customer entities. These are mapped into: `List<Customer>` and returned to the service.
7. Service converts Entity → ResponseDTO (id, customerCode, fullName, email, phone, address, status, createdAt).
8. Controller returns ResponseEntity with the list
9. Spring Boot converts DTO list → JSON

10. Frontend (Thunder Client) receives the JSON and displays filtered results

Test 7: Validation Error

The screenshot shows the Thunder Client interface. On the left, a list of requests is visible, with the most recent one being a POST request to `localhost:8080/api/customers` at 19 mins ago. The main panel displays the details of this request. The 'Body' tab is selected, showing the JSON content: `{ "customerCode": "C", "email": "invalid-email" }`. The 'Response' tab is also visible, showing a 400 Bad Request status with a detailed error message: `{ "timestamp": "2025-12-02T09:25:12.7382369", "status": 400, "error": "Validation Failed", "message": "Invalid input data", "path": "/api/customers", "details": { "customerCode: Customer code must be 3-20 characters", "email: Invalid email format", "customerCode: Customer code must start with C followed by numbers", "fullName: Full name is required" } }`. The bottom panel shows the terminal output, which contains the raw JSON response.

Explanation of the code flow of the validation:

1. Browser → POST `/api/customers` (or PUT `/api/customers/{id}`)
 - The user sends customer data from the frontend.
2. CustomerRestController receives request (`@PostMapping` or `@PutMapping`)
 - Spring Boot detects the route and maps the JSON to CustomerRequestDTO.
3. Spring automatically applies `@Valid` on CustomerRequestDTO
4. Validation fails → Spring throws `MethodArgumentNotValidException`
5. GlobalExceptionHandler catches `MethodArgumentNotValidException`
6. Handler extracts all field validation errors. For example, `fullName`: must not be blank, `email`: must be a valid email
7. Handler builds `ErrorResponseDTO`.
 - Includes status, message, path, and list of error details.
8. Handler returns `ResponseEntity<ErrorResponseDTO>` with 400 BAD REQUEST
9. Spring converts `ErrorResponseDTO` → JSON automatically
10. JSON error response sent to browser
 - Frontend shows validation messages to the user.

Test 8: Resource Not Found

The screenshot shows the Thunder Client interface. On the left, a list of requests is visible, with the selected request being a GET to `localhost:8080/api/customers/999`. The main panel shows the details of this request. The status is **404 Not Found**, with a size of 201 Bytes and a time of 19 ms. The response body is a JSON object:

```
1 {
2   "timestamp": "2025-12-02T09:25:47.6684629",
3   "status": 404,
4   "error": "Not Found",
5   "message": "Customer not found with id: 999",
6   "path": "/api/customers/999",
7   "details": null
8 }
```

At the bottom, the terminal shows a log entry:

```
2025-12-02T09:25:47.6684629 WARN 1816 --- [customer-api] [nio-8080-exec-9] .m.m.a.ExceptionHandlerExceptionHandlerResolver : Resolved [com.example.customer_api.e
xception.ResourceNotFoundException: Customer not found with id: 999]
```

Explanation of the code flow of the unknown customer:

1. Browser → GET `/api/customers/999`. The frontend requests a customer with ID 999 (which does not exist).
2. `CustomerRestController` receives request (`@GetMapping("/{id}")`). Spring Boot maps the URL and extracts `id = 999`.
3. Controller calls `customerService.getCustomerById(id)`. Passes the ID to the service layer.
4. Service calls `customerRepository.findById(id)`. JPA tries to find the customer in the database.
5. JPA automatically queries database. SQL executed: `SELECT * FROM customers WHERE id = 999;`
6. Repository returns `Optional.empty()` (customer not found). No entity exists with that ID.
7. Service throws new `ResourceNotFoundException("Customer not found with id: 999")` because the optional is empty.
8. Exception goes to `GlobalExceptionHandler`. Spring detects a custom exception and transfers control to: `@ExceptionHandler(ResourceNotFoundException.class)`
9. `GlobalExceptionHandler` builds `ErrorResponseDTO` including `status = 404`, `error = "Not Found"`, `message = "Customer not found with id: 999"`, and `path = "/api/customers/999"`.
10. Handler returns `ResponseEntity<ErrorResponseDTO>` with 404 NOT FOUND. Standard JSON error is created.
11. Spring converts `ErrorResponseDTO` → JSON automatically
12. JSON error response sent to browser.

Test 9: Duplicate Resource

The screenshot displays the Thunder Client interface. On the left, a list of requests is shown, with the selected request being a POST to `localhost:8080/api/customers`. The main panel shows the request details: Method (POST), URL (`http://localhost:8080/api/customers`), Headers, Auth, Body (JSON), Tests, and Pre Run. The JSON body contains:

```
{
  "customerCode": "C007",
  "fullName": "Test User",
  "email": "john.doe@example.com"
}
```

The response panel shows a status of 409 Conflict, size of 207 Bytes, and time of 25 ms. The response body is:

```
{
  "timestamp": "2025-12-02T09:26:28.0363002",
  "status": 409,
  "error": "Conflict",
  "message": "Email already exists: john.doe@example.com",
  "path": "/api/customers",
  "details": null
}
```

The terminal at the bottom shows a SQL query and a warning message:

```
select
  c1_0.id
from
  customers c1_0
where
  c1_0.email=?
limit
  ?
```

2025-12-02T09:26:28.038+07:00 WARN 1816 --- [customer-api] [nio-8080-exec-1] .m.m.a.ExceptionHandlerExceptionHandlerResolver : Resolved [com.example.customer_api.e
xception.DuplicateResourceException: Email already exists: john.doe@example.com]

Explanation of the code flow of the duplication:

1. Browser → POST `/api/customers`. Frontend (Thunder Client) sends new customer data.
2. `CustomerRestController` receives request (`@PostMapping`). Spring Boot maps JSON → `CustomerRequestDTO`.
3. Controller calls `customerService.createCustomer(requestDTO)`. Delegates processing to the service layer.
4. Service checks if customerCode or email already exists:
`customerRepository.existsByCustomerCode(requestDTO.getCustomerCode())`
5. Repository queries the database. JPA executes `SELECT COUNT(*) FROM customers WHERE customer_code = 'C001';`
6. Repository returns true (duplicate found). Meaning the code is already used by another customer.
7. Service throws new `DuplicateResourceException("Customer code already exists")`. This stops the normal flow immediately.
8. Exception is sent to `GlobalExceptionHandler`. Spring automatically routes it to:
`@ExceptionHandler(DuplicateResourceException.class)`
9. `GlobalExceptionHandler` builds `ErrorResponseDTO` including `status = 409`, `error = "Conflict"`, `message = "Customer code already exists"`, `path = "/api/customers"`.
10. Handler returns `ResponseEntity<ErrorResponseDTO>` with 409 CONFLICT. Standardized JSON error prepared.
11. Spring Boot converts `ErrorResponseDTO` → JSON
12. JSON error response sent back to browser.